

Name: Rhilo Sotto
RED ID: 130551574
Date: 11/22/2024

Lab 8 Report

1. What does your program do?

The program uses EEPROM to keep track of current duty cycle and to increase duty cycle by 10% up to 100% when powering on the microcontroller, cycling back to 0%.

The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost.

2. What variables/registers does your program use?

- **dutyCycle** (global uint8_t array)
- **dutyCycleIndex** (uint8_t variable)
- **address** (uint8_t EEMEM variable)
- **DDRB** (register)
- **TCCR0A** (register)
- **OCR0A** (register)
- **OCR0B** (register)
- **TIMSK0** (register)
- **TCCR0B** (register)
- **PORTB** (register)

3. What is the main algorithm/logic of your program?

The main program calls the **init** function to initialize the DDRB register to set the LED on PINB5 as an output.

Then it calls the **timer_init** function to set Timer 0's mode to CTC which will cause the clock to reset upon reaching the value stored in OCR0A. TCCR0B is used to set the prescaler value to x1024 and start the timer, which was chosen since the required interrupt period given is 5ms. This can be achieved with a max period of 16.384 ms.

The program then uses an address in EEPROM allocated by the EEMEM attribute, to read a byte in EEPROM that corresponds to the last saved duty cycle. If this byte is the default EEPROM value of 0xFF, then it sets the index to 0 and updates the EEPROM byte to 0, else it sets the index to the next value mod 11, which will cycle the index back to 0 after it reaches 11, and updates the EEPROM byte to the new index. OCR0B is set to the mapped duty cycle corresponding to the duty cycle index in the duty cycle array (0%, 10%, ... 100%) which is calculated beforehand.

Global interrupts are then enabled in main.

The program enters the infinite while loop.

The program has two interrupt service routines.

1. **ISR (TIMER0_COMPA_vect)** is Timer0's compare register A interrupt, which turns on the LED.
2. **ISR (TIMER0_COMPB_vect)** is Timer0's compare register B interrupt, which turns off the LED.

4. What external hardware connections did you make?

The LED is onboard, wired to PB5.

Appendix (Code)

```
/*
 * Lab8.c
 *
 * Created: 11/15/2024 2:40:05 PM
 * Author : rnsot
 */

#define F_CPU 16000000UL // 16MHz clock frequency

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/eeprom.h>

uint8_t dutyCycle[] = {0, 8, 15, 23, 31, 39, 46, 54, 62, 69, 77};
uint8_t EEMEM address = 0xFF; // default value at EEPROM address

void init(void) {
    DDRB |= (1 << DDRB5); // output to LED
}

void timer_init(void) {
    // F_CPU = 16MHz, want 5ms
    // TIMER 0 - interrupt timer (5ms)
    // ps = 1024, NM -> 16384us = 16.384ms **
    TCCR0A |= (1 << WGM01); // CTC Mode
    OCR0A = 77; // timer period (5ms/16.384ms * 256 time steps = 78.125 time steps - 1 =
77 OCRA)
    TIMSK0 |= (1 << OCIE0A) | (1 << OCIE0B); // set ISR COMPA and COMPB vector
    TCCR0B |= (1 << CS02) | (1 << CS00); // Pre-scaler set to 1024, STARTS timer0
}

int main(void) {
    init();
    timer_init();

    uint8_t dutyCycleIndex = eeprom_read_byte(&address);
    if (dutyCycleIndex == 0xFF) { // default EEPROM value is 0xFF
        dutyCycleIndex = 0;
        eeprom_update_byte(&address, 0);
    }
    else {
        dutyCycleIndex = (dutyCycleIndex + 1) % 11;
    }
}
```

```

        eeprom_update_byte(&address, dutyCycleIndex);
    }
    OCR0B = dutyCycle[dutyCycleIndex];

    sei();
    while (1) {}
}

ISR (TIMER0_COMPA_vect) { // Timer0 compare register A interrupt
    if (OCR0B != 0) PORTB |= (1 << PORTB5); // HIGH, LED ON
}

ISR (TIMER0_COMPB_vect) { // Timer0 compare register B interrupt
    if (OCR0B != OCR0A) PORTB &= ~(1 << PORTB5); // LOW, LED OFF
}

```